SightLine — evidence sheet

Repository: github.com/adeviwon/SightLine · commit `2e12860` (branch `pwa`) · 2026-10-22 · Imperial Hackathon Hong Kong & Macau 2026

One page, one purpose: every claim we make, and the file in this repository that contains it. Where a number was not already written down, it was computed from the artifact with the repository's own scoring code — the command is in §5, and it prints what is printed here.

1. What it is

SightLine is an installable web app (PWA) that photographs a document, recognises the text and reads it aloud — for people who cannot read the page, including people with visual impairment. It runs entirely on the phone: after the first install there is no network call, no upload, and no OCR library — recognition is our own 942,166-parameter CRNN+CTC network (37-symbol alphabet, digits/letters/space) shipped as a self-contained ONNX graph, with a build gate that fails the build if a third-party OCR engine reappears.

What runs where (each figure with its source).

Item	Value	Source in this repo
Live app (open on a phone, then add to Home Screen)	https://adeviwon.github.io/SightLine/	`README.md`
Recogniser	942,166 parameters, 37 symbols, CTC; held-out crop word accuracy 28.0% on 7,875 crops	`artifacts/ocr_training.json`; `docs/14_HACKATHON_BENCHMARK.md`
Installed payload	26.45 MiB of models + tokenizer, self-contained; ≈37.2 MiB installed once the 10.73 MiB onnxruntime wasm is counted; 0 bytes leave the phone	`docs/11_JUDGE_TECHNICAL_DEEP_DIVE.md`; `docs/13_ECC_CODE_REVIEW.md` (`bundle-check`)
Where inference happens	In the phone's browser, in WASM (onnxruntime-web), from the service-worker cache; after install the app never reaches the network	`app/sw.js` (`caches.match`, cache-only); `docs/PRIVACY.md`
Desktop layout analysis (where available)	Optional, off-device, and not part of the phone path; the phone path is the one measured below	`docs/01_ARCHITECTURE.md`
Build gate against vendored OCR	Build fails if Tesseract or another OCR library returns to the bundle	`ml/src/bundle_check.py` via `bash run.sh bundle-check`; `HANDOFF.md` §6

2. The measured results

What the bench is. `bash run.sh bench-pipeline` drives the real app — `app/`, unmodified — in Playwright **WebKit**, the engine iOS Safari uses, over **12 rendered documents** (3 document types × 4 capture profiles: `studio_clean`, `handheld_light`, `off_axis`, `jpeg_social`), and writes one raw record per sample. `ml/src/bench_hackathon.py` then scores those records — the browser computes no metrics, so the two cannot drift apart. No sample is excluded: all 12 profiles are in the headline, 0 errors. **Scope of the shipped figure: 12 synthetic rendered documents, WebKit, one record per document, on the current capture-profile fixtures.**

The headline, stage by stage: four defects fixed, four measurements — then the fixtures themselves

Each row is a frozen artifact from the point in the work where that fix landed. Same 12 documents, same metric, same engine. **The first four rows were measured on the previous fixture generation** (the bench images predated the capture-profile re-tune); the last row is the app today, on the current fixtures. The two generations are labelled, not merged.

Headline end-to-end numbers. Word accuracy and CER are the definitions in `ml/src/bench_hackathon.py` (word accuracy: fraction of ground-truth words present in the read, after `norm()`; CER: (substitutions + insertions + deletions) / ground-truth characters on normalised text).

Stage	Word acc.	CER	Expected fields recovered	Docs emitting ≥1 field	Classif.	Errors	Artifact
Start of this work (fixed 256px graph, no ranking raw arm, deskew sign inverted) — *previous fixtures*	3.6%	0.708	0 / 24	3	33.3%	0	`artifacts/pipeline_raw.json`
+ dynamic-width graph (padding no longer leaks through the BiLSTM) — *previous fixtures*	17.1%	0.600	1 / 24	4	58.3%	0	`artifacts/pipeline_raw_dynamic.json`
+ the raw image is back in the OCR candidate list — *previous fixtures*	35.0%	0.465	4 / 24	7	66.7%	0	`artifacts/pipeline_raw_rawarm.json`
+ deskew no longer doubles the tilt (what shipped at the time) — *previous fixtures*	45.9%	0.303	5 / 24	9	83.3%	0	`artifacts/pipeline_raw_deskew.json`
+ the retrained	57.2%	0.217	8 / 24	10	91.7%	0	`artifacts`

Stage	Word acc.	CER	Expected fields recovered	Docs emitting ≥1 field	Classif.	Errors	Artifact
model on the current, non-stale fixtures (**what ships today**)							`/pipeline_raw_shipped.json`

Two fixture generations, and the A/B that separates them. The four earlier rows were measured on the **previous** capture-profile fixtures (rendered 2026-10-20 07:37, before `CAPTURE_PROFILES` was re-tuned at 23:54); the last row is the same 12 documents re-rendered with the current profiles and scored with the shipped retrain3 model. They are **not directly comparable**: a controlled A/B — the old graph, re-benched on the current fixtures — reads **47.7% / 0.278 / 20.8% / 91.7%** (`artifacts/pipeline_raw_oldmodel_currentfix.json`), so of the 45.9% → 57.2% move **+1.7 points is the benchmark getting more realistic and +9.5 is the retrain** — never one model gain (`docs/14_HACKATHON_BENCHMARK.md`, which carries the 2×2 table and the `ml/tests/test_fixture_freshness.py` gate that now prevents a stale bench).

Reading the two field columns: “expected fields recovered” counts how many of the 24 ground-truth values (12 documents × 2) the app actually got right — a value counts when it matches exactly, or when it is a masked suffix of the expected one (the app is allowed to mask sensitive values) — rule: `bench_hackathon._recovered`, which is the rule `bash run.sh bench-hackathon` prints. “Docs emitting ≥1 field” counts records that produced any field value at all, right or wrong: at the starting stage 3 documents emitted 4 values and **all four were wrong**, which is the distinction that column exists to keep visible.

Per-profile breakdown

Same four artifacts, split by capture profile. Word accuracy / CER, fields recovered, and document classification per profile (3 documents per profile).

Profile	Stage	Word acc.	CER	Fields	Classif.	Artifact
`studio_clean`	start	10.3%	0.436	0 / 6	66.7%	`pipeline_raw.json`
	dynamic	49.9%	0.236	1 / 6	100%	`pipeline_raw_dynamic.json`
	raw arm	52.3%	0.225	2 / 6	100%	`pipeline_raw_rawarm.json`
	deskew *(prev. fixtures)*	52.3%	0.225	2 / 6	100%	`pipeline_raw_deskew.json`
	shipped · current fixtures	60.1%	0.169	2 / 6	100%	`pipeline_raw_shipped.json`
`handheld_light`	start	1.7%	0.812	0 / 6	0%	`pipeline_raw.json`
	dynamic	1.2%	0.760	0 / 6	33.3%	`pipeline_raw_dynamic.json`
	raw arm	46.5%	0.376	1 / 6	66.7%	`pipeline_raw_rawarm.json`
	deskew *(prev. fixtures)*	46.5%	0.376	1 / 6	66.7%	`pipeline_raw_deskew.json`
	shipped · current fixtures	75.2%	0.075	4 / 6	100%	`pipeline_raw_shipped.json`
`jpeg_social`	start	0.9%	0.732	0 / 6	33.3%	`pipeline_raw.json`
	dynamic	16.4%	0.523	0 / 6	66.7%	`pipeline_raw_dynamic.json`
	raw arm	39.8%	0.378	1 / 6	66.7%	`pipeline_raw_rawarm.json`
	deskew *(prev. fixtures)*	39.8%	0.378	1 / 6	66.7%	`pipeline_raw_deskew.json`
	shipped · current fixtures	43.7%	0.369	2 / 6	66.7%	`pipeline_raw_shipped.json`
`off_axis` (4° tilt + perspective)	start	1.6%	0.854	0 / 6	33.3%	`pipeline_raw.json`
	dynamic	1.1%	0.882	0 / 6	33.3%	`pipeline_raw_dynamic.json`
	raw arm	1.1%	0.882	0 / 6	33.3%	`pipeline_raw_rawarm.json`
	deskew *(prev. fixtures)*	45.1%	0.234	1 / 6	100%	`pipeline_raw_deskew.json`
	shipped · current fixtures	49.6%	0.255	0 / 6	100%	`pipeline_raw_shipped.json`

The `handheld_light` and `jpeg_social` rows are unchanged between the last two stages, which is the expected shape: those profiles sit at ~0° skew, where `estimateSkew()` returns exactly 0 and the deskew fix is unreachable (`app/js/pipeline.js`, “It is

unreachable on the working profiles, which is why it went unnoticed").

What this bench proves, and what it does not

Proves: the shipped app's entire path — preprocess, line detection, recognition, field extraction, classification — on these 12 rendered documents, in WebKit, reproducibly (`bash run.sh bench-pipeline` then `bash run.sh bench-hackathon`).

- **It is not a generalisation claim.** The recogniser never trained on these renders, but the classifier's templates *are* in its corpus, so its end-to-end agreement here is not held-out evidence; the classifier's held-out figure is 95.8% accuracy / 95.7% macro-F1 over 5 seeds on n=24 documents, and it carries its own caveat — a near-perfect score on 24 documents means the task is easy, not that the model is strong (`artifacts/step3_classifier_metrics.json`, `docs/14_HACKATHON_BENCHMARK.md`).
- **Word accuracy is bag-of-words containment, not reading order.** It counts how many ground-truth words appear anywhere in the read text after `norm()`. A scrambled read can score high on it. Treat it as recall of vocabulary, not as comprehension.
- **12 documents is small (3 per profile).** No per-profile difference here is statistically resolvable, and no confidence interval is claimed on n=3. The per-profile table is a description, not an inference.
- **The hard case is not in this table.** The app was also measured with the whole-page Python harness on hand-photographed pages: on the **20-page** set, detector recall **67.9%**, line-exact **1.8%**, CER **56.5%**; the **193-page** set-B reproduction of the shipped checkpoint reads line recall **71.6%**, line-exact **2.4%**, CER **58.4%**. Both are real, on different page sets — and either way it is the honest number for real photographs, far worse than anything above (`docs/14_HACKATHON_BENCHMARK.md`; `docs/03_RESULTS.md` §3; `HANDOFF.md` §3).
- **These artifacts are frozen copies.** Re-running `bash run.sh bench-pipeline` overwrites `artifacts/pipeline_raw.json`; the stage files (including `pipeline_raw_shipped.json` for the current fixtures) were copied aside at the moment each fix landed, so the progression stays readable. Re-running can only differ because the app changed — which is the point of publishing it. The fixture generation of every row is labelled, and `ml/tests/test_fixture_freshness.py` now fails if the bench images stop matching the capture profiles.

3. What we got wrong and fixed

Four train/serve contract defects. Each one is a case where what the app fed the model was not what the model was trained on, and each one was found by measurement, not by reading the code. In this project the input contract has outranked the model every time (`docs/18_CRNN_ACCURACY_STRATEGIES.md` §0).

The four defects, in the order they were found, with the magnitude each one was measured to have.

Defect	What was actually wrong	Measured magnitude	Source
(a) The exported graph was a fixed 256px	Training built every crop at its natural width clamped to `MAX_W = 320`, and the corpus median *is* that clamp (of 3,270 held-out crops only 40 are ≤256px). The ONNX graph was baked at 256, so the app fed the model the first 256px of nearly every line — the tail was cut off before the model ever saw it.	Crop-level exact match **44.7% at natural width vs 0.0% cut to 256**; the read changed on **200 of 200** crops (`PATIENT JANE DOE DOB 04021979` → `PERIENT JANE DOE 2008010`; `DATE 01102026` → `DATE 01020`). End-to-end, same 12 documents: word accuracy **3.6%** → **22.4%**, mean CER **70.8%** → **57.5%**, read tokens **468** → **680 (+45%)**. The graph is now exported at the training clamp, and the contract test asserts *equality* with it — the old assertion was `width <= MAX_W`, which any narrower graph satisfies, and it passed hardest on the worst value.	`docs/14_HACKATHON_BENCHMARK.md`; `app/js/recognizer.js`; `ml/src/export_onnx.py`; `ml/tests/test_graph_width_contract.py`; `artifacts/pipeline_raw_final.json`
(b) Right-padding leaked through the backward pass	The CRNN carries a two-layer *bidirectional* LSTM, whose backward pass starts at the right edge — inside the padding. CTC blanks the padded region for the loss, so training never noticed; at serve time the padded zeros moved the hidden states across the valid frames.	Padded serve path **19.0% crop-level exact / CER 0.308** against **30.5% / 0.198** at natural width, agreeing with the natural decode on only **43%** of crops; measured max |delta| 9.10 at content 64 and 15.28 at 200 (logit units), enough to change the decoded string. 74.7% of held-out crops are padded at all against the 320 clamp. Fixed by exporting a dynamic-width graph; padding is now a known loss, not a neutral default.	`app/js/recognizer.js`; `ml/src/export_dynamic.py`; `ml/tests/test_padding_invariant.py`; `docs/18_CRNN_ACCURACY_STRATEGIES.md` §0.1
(c) The OCR candidate list *replaced* the preprocessed list	With the restorer on, the app substituted the restorer's three candidates for the preprocessed ones — and none of those three is the raw image. So the ranking could only choose among three degraded variants of each page and never saw the unprocessed deskewed image at all. The ranking rule was innocent: raw wins on the rule's own objective once it is allowed to compete.	On the app's own 12 documents, 145 paired crops, with the crop set, crop geometry and model held fixed and only the page transform varied: **raw 45.5% exact line match** vs **11.0% for the arm it was actually picking** (tile-CLAHE 40.7%, CLAHE+Otsu 13.8%). The shipped ranking picked one of the three restored arms on **12 of 12** documents, costing **34.5 points** of exact line match, **29.6 points** of word accuracy and **9 of 24** fields — while not even winning on `structure_score`, the ranking's own objective (103 tokens against raw's 146). Fixed by prepending	`app/js/app.js` (comment and the one-line fix); `app/js/pipeline.js` (same fix for the non-restorer path)

Defect	What was actually wrong	Measured magnitude	Source
		the raw candidate; the rule was left alone.	
(d) Deskew applied its own estimate with the sign inverted	`rotate()` takes the angle in the opposite sense to the one `estimateSkew()` measures, so applying the estimate directly rotated the page *away* from straight and doubled the tilt. The estimator was never wrong: it returned the right magnitude, and re-estimating the app's own deskewed output returned 6.5–9.0°. Preceding that, the estimator scored each angle by a raw count of dark samples, which is not comparable across angles and left every tie to the sweep's first angle — a blank page scored −10, an axis-aligned page −8, a 4° page −7. It now uses a normalised projection-profile score, and the gate that would have caught the sign is a *round trip* (apply, re-estimate, require ~0), not “is the angle plausible”.	`off_axis` word accuracy **1.1%** → **45.1%** (start → today); the old expression destroyed the two pages whose raw detection was already marginal (legal 3 → 0 bands, prescription 1 → 0). Bench-wide: word accuracy **35.0%** → **45.9%**, CER **0.465** → **0.303** *(previous fixture generation — see §2's label; the current-fixture equivalent of the deskew step is the A/B control row, 47.7%)*.	`app/js/pipeline.js` (the negation and the measured off_axis table, both in the comment); `app/tools/deskew_verify.js` (the round-trip assertion); `app/tools/offaxis_detect.js` (records the 1.1% starting state and the shear-sign finding); `HANDOFF.md` §4 item 0; values computed from `artifacts/pipeline_raw_rawarm.json` and `artifacts/pipeline_raw_deskew.json`

Full detail, including the measurements that were tried and reverted, is in `docs/18_CRNN_ACCURACY_STRATEGIES.md` and `docs/14_HACKATHON_BENCHMARK.md`; the commit log is the primary record. Nothing in this section is a claim about the model — every one of the four was an interface defect between training and serving.

4. Why not Tesseract or Google Lens

The differentiation we claim is architectural and verifiable, not a benchmark win:

- **On-device by construction.** There is no upload target, because there is nowhere to upload to: the service worker serves from cache only, inference runs in WASM on the phone, and after install the app works in airplane mode. A cloud OCR service (Google Lens and friends) cannot make that claim — the page image leaves the device.
- **No vendored OCR engine.** Tesseract was removed; the recogniser is a network we trained ourselves. `bash run.sh bundle-check` fails the build if a third-party OCR engine comes back — a gate, not a promise (`ml/src/bundle_check.py`; `docs/13_ECC_CODE_REVIEW.md`).
- **Refusal over guessing.** The app is built to say it could not read something rather than assert a value it is not sure of (see §6). That is the opposite of the default behaviour of a general OCR service, which returns its best guess with no notion of “do not act on this”.
- **Weights and algorithm we can read end to end.** 942,166 parameters, 37 symbols, a self-contained ONNX graph, an in-repo export and a parity test against the browser decoder.

And the honest part, because a judge will ask it — now measured, not asserted. `ml/src/tesseract_compare.py` ran **Tesseract 4.1.1** (leptonica-1.82.0, `eng` only, default LSTM, CLI) and our shipped graph over the *same* 12 documents, the *same* detected crops and the *same* ground truth: 188 scored lines (`docs/24_ENGINE_HEAD_TO_HEAD.md`; `artifacts/engine_head_to_head.json`). These figures are a **re-measurement on the current, regenerated fixture generation** (2026-10-22; the previous generation's rows are kept at `artifacts/engine_head_to_head_oldfixtures.json` and they were worse for Tesseract — see the withdrawn claim below). Every figure below is the artifact's, unrounded:

Same documents, same pixels, same ground truth — only the engine varies. Tesseract's winning rows stay in, at the top, where they belong.

arm	line exact	line exact (punct-blind)	word acc.	CER	fields
Tesseract — whole page, `--psm 3` (its fair integration)	95.7%	99.5%	98.9%	0.000	100.0%
Tesseract — native-resolution line crops (its fair input)	86.7%	93.1%	96.8%	0.007	95.8%
Tesseract — the *identical* 32 px crops we read (unfair to Tesseract, reported anyway)	22.9%	31.9%	68.0%	0.183	45.8%
ours — CRNN on 32 px crops, raw deskewed page (what ships)	10.6%	48.9%	67.2%	0.152	75.0%

Field recovery is over 24 ground-truth values (2 per document) — on that column a one-field lead is noise, not a win. Word accuracy and CER are the repository's own definitions (`ml/src/evaluate.py`, `ml/src/bench_hackathon.py`). Line crops and whole pages are quoted separately because they are different integrations, not because one flatters us.

- **Tesseract wins the aggregate, and it wins every profile, and we say that first.** 95.7% of lines exactly against our 10.6%, CER 0.000 against 0.152, 98.9% word accuracy against 67.2%. On clean, flat scans it is the better reader, and no measurement of ours will change that (`docs/24_ENGINE_HEAD_TO_HEAD.md` §3 and §5 state the same thing).
- **The fairness correction — stated, not hidden.** Our alphabet is 37 symbols (0-9, A-Z, space): it *structurally cannot emit punctuation*. 120 of the 188 scored rows contain punctuation, so they can never be a string-exact match however correct the read is — that alone is the whole distance between 10.6% and 48.9% line exact. On the identical-pixel control, punctuation-blind, ours takes **49 lines Tesseract misses and loses 17** (`artifacts/engine_head_to_head.json`, `paired.identical_pixels_ours_vs_tess_arm_a_punct_blind`). The strict column is not withdrawn — it is what an engine with no punctuation gets, and it is printed beside the blind one.
- **The hand-held claim, withdrawn.** Earlier versions of this sheet and of slide 7 quoted Tesseract at **12.4% word accuracy on** `handheld_light` and claimed a 5.5× win for us on real captures. That profile was rendered 17–67× harsher than a real photograph. On the regenerated fixtures the same three documents read **98.2%** (native line crops) and **98.6%** (whole page) at CER 0.000 with every field recovered, against our **78.4%** and 83.3% of fields. The other three profiles are pixel-identical between the two generations and Tesseract's reads on them are unchanged, so the 12.4% was the simulator and not the engine. Stated as a measurement, not softened into a caveat.
- **The safety fact.** On the clean page — pixels unchanged between the two generations — Tesseract read `Rx Amoxicillin 500mg capsules` as `RX AMOXICILLIN 300MG CAPSULES`: a plausible-word substitution of the dosage, on the exact field this product exists to protect. Its language model makes a wrong read still look like text, so nothing in the output says which word is the invention. Ours emits no character outside its 37-symbol alphabet (0.0% punctuation in its output), drops no line on any profile, and the app refuses a field it cannot verify (§6).

Plainly: **on these 12 documents Tesseract is the stronger reader on every profile, and we print that first. What we claim instead is narrower and measured: our 37 symbols cannot invent a punctuation mark or a digit, and the app refuses rather than guesses.** We do *not* claim to beat Tesseract overall, and we no longer claim the hand-held profile. The decision this measurement supports is *which* engine for *which* product: a vendored engine means a second WASM runtime plus its language data — `eng.traineddata` 4,113,088 B and `osd.traineddata` 10,562,727 B on the box it was measured on — and the build gate refuses to let an OCR library into the bundle (`docs/24_ENGINE_HEAD_TO_HEAD.md` §5; `ml/src/bundle_check.py`).

Superseding note: the earlier comparison recorded in this repository — Tesseract field accuracy 62.2% against our 7.4% at the best `evaluate.py` stage (`docs/14_HACKATHON_BENCHMARK.md`, “The honest comparison a judge should see”; `artifacts/eval_results.json`) — was measured before the recogniser was retrained and before the four train/serve defects in §3 were fixed, so it is misleading in both directions. The table above is its replacement, and it is the one to quote. Re-derive it with `./.venv/bin/python ml/src/tesseract_compare.py` — 12 documents, 188 ground-truth lines, no number on this sheet taken on trust.

5. How to verify us in three steps

1. **Open the live app** — https://adeviwon.github.io/SightLine/ — on a phone, and add it to the Home Screen. It installs.

2. **Turn on airplane mode.** Nothing about the app changes: there is no network call left to make (`docs/PRIVACY.md` §offline, §airplane mode).
3. **Photograph a document and listen.** You should hear the recognised text read aloud. We also suggest including one deliberately hard photo, where the app *refuses* — "could not read that, please check with a pharmacist". That refusal is the design, not a bug (`docs/13_ROUND1_SUBMISSION_KIT.md` §demo run sheet).

The two-minute repository check

```
# 1. score the raw pipeline records already in the repo (reads artifacts/, ~10 s, no browser)
bash run.sh bench-hackathon
#    prints the Imperial step-3 block: classifier 95.8% (n=24, 5 seeds),
#    recogniser 28.0% held-out crop word accuracy, and the shipped-app end-to-end line.
#    It NAMES anything it could not measure and exits non-zero instead of defaulting it to zero.

# 2. re-drive the app yourself (Playwright WebKit = iOS Safari's engine, ~4 min)
bash run.sh bench-pipeline     # overwrites artifacts/pipeline_raw.json
bash run.sh bench-hackathon    # score what you just produced

# 3. the unit gates the fixes are pinned by (each prints "N/N checks passed" and exits non-zero or
node app/tools/crop_contract.js         # crop geometry (9 checks)
node app/tools/recognizer_parity.js     # browser CTC decoder vs the Python reference
node app/tools/warning_gate.js          # the honesty invariants, on all 12 samples
```

And the bridge between this sheet and the artifacts — this exact command reproduces every number in §2 from a stage artifact, using the repository's own metric functions (imported, not re-implemented):

```
./.venv/bin/python -c "import sys,json;sys.path.insert(0,'ml/src');import bench_hackathon as B;pri

# prints:
# {'n': 12, 'errors': 0, 'field_recovery': 0.3333333333333333, 'classification': 0.916666666666666
#  'word_accuracy': 0.5715791133599353, 'cer': 0.21709805749692557}
# -> 57.2% word accuracy, CER 0.217, 8 of 24 expected fields, 91.7% classification, 12 documents,
#    (current fixture generation, shipped retrain3 model)
# Swap in 'artifacts/pipeline_raw_deskew.json' for the previous fixture generation:
# -> 45.9% word accuracy, CER 0.303, 5 of 24 expected fields, 83.3% classification
```

6. The safety design

This app reads things people will act on — a dose, an account number, a date. A phone app must not be the last check on a dose: the consequence of an invented "500 mg" is a person taking the wrong amount, and nobody in the loop is in a position to catch it. So the design rule is **refusal over guessing**: when the app cannot read a document, it says so and asserts nothing. It still reports what it saw as text; it does not invent a value.

The measurement that forced the design. A confidence threshold is the obvious gate, and it does not work here. On a pure-speckle page (no text at all) the recogniser emitted glyph-like text — "EAX CE E 0RO 121G000 …" — at **0.678 confidence**, *inside* the 0.687–0.974 band that real reads occupy, and the invented value it produced passed every shape validator: it was a legal 9-digit account number and a legal identifier. Neither available signal separates a hallucination from a read. What does separate them is **document evidence**: the speckle page scores 0 on every document keyword, while real reads score 1–17 even when they classify as the wrong type. So a read with no document evidence asserts no fields — the gate is on evidence, not on confidence. Measured cost of that gate on the 12-document bench: 1 of 12 documents affected, and it surfaced no fields anyway.

- Rule and defect: `app/js/pipeline.js` ("A FIELD IS ONLY MEANINGFUL INSIDE A DOCUMENT TYPE"), the document-evidence gate, plus the follow-on fix that a model override only bypasses the gate when the model is confident (≥0.6) — a weak model opinion about a text with no document keyword is not evidence.
- The same finding, recorded: `docs/18_CRNN_ACCURACY_STRATEGIES.md` §0.1 ("Confidence cannot gate correctness", "What DOES separate a hallucination from a read is document EVIDENCE"); the band 0.687–0.974 is also the confidence range of real reads in `artifacts/pipeline_raw_final.json`.

- Pinned by a gate, not a comment: `node app/tools/warning_gate.js` asserts the honesty invariants on all 12 samples, and `node app/tools/e2e_paths.js` covers the refusal path (`HANDOFF.md` §6).
- No data leaves the device, so the safety story has no privacy side-condition: `docs/PRIVACY.md`; `app/sw.js` serves from cache only.

7. The policy anchor — and why it is the 2025 edition

The Open Challenge rule names one edition, verbatim: the problem must be *"explicitly anchored in a priority from either the Hong Kong or Macau **2025** Policy Address that is not already covered by the five prescribed challenges"* (official FAQ, read 2026-10-21). So the **2025** edition leads this sheet — every anchor below is quoted verbatim from the official 2025 text with that text's own paragraph number — and the **2026** edition is presented as the funded *follow-through*, never as the anchor. Full provenance, both PDFs and their SHA-256s: `docs/19_POLICY_ANCHOR.md` §0.1–§0.2; the 2025 PDF is policy-full_en.pdf, 118 pp, sha256 `c4ecaf20…18f3ec`.

The three 2025 anchors, each mapped to a feature that already exists in the app — not to an aspiration.

¶ (2025)	The priority, as printed	What it anchors in SightLine
¶75	AI application: *"…render carer support through an AI chatbot to give carers more convenient access to information"*; generative AI in clinical workflows	a carer or patient getting information from the phone they already own, with no network — the exact use case
¶227	Healthcare reform for an ageing society, incl. *"enhancing ophthalmic services"* and *"a new high-capacity ambulatory cataract surgery centre"* (¶226 opens the healthcare section, so the ophthalmic measures sit in ¶227)	the only directly vision-facing commitment in either edition
¶266	Working Group on Ageing Society Strategies — *"…elderly care, healthcare, housing, culture and leisure, and gerontechnology"*; *"by 2046, people aged 65 and above will account for 36% of our total population"*	gerontechnology for older people is the group's named remit; the 36% is the growth case

The 2026 edition is the follow-through — the money, not the anchor. ¶113 allocates **$300 million** to the "AI for Welfare Lab" (WelfareHK.ai) and ¶398–400 earmark **$100 million** for the Gerontechnology Promotion Scheme, with ¶398 letting subsidised welfare units procure or rent gerontechnology — a paying buyer for a 26.7 MB installable PWA (`docs/19_POLICY_ANCHOR.md` §4). The two editions are *not* numbered alike: the 2026 items this repository originally cited — 2026 ¶108 (AI in healthcare) and ¶113 — do not exist as those items in the 2025 text, which is exactly why the 2025 paragraph numbers above are the ones on the slide and in the proposal (`docs/13_ROUND1_SUBMISSION_KIT.md`, policy-anchoring section).

Mapping strength, stated for the Q&A rather than discovered in it: ¶75 and ¶266 are strong (named AI-for-carers text; gerontechnology named as a remit), ¶227 is strong and vision-specific, and the stretched mappings this project deliberately does *not* use — 2026 ¶377 (rehabilitation *places*), ¶437 (internal government AI projects), ¶441–442 (critical-infrastructure cybersecurity) — are named as stretches in `docs/19_POLICY_ANCHOR.md` §6. Neither edition names the visually impaired anywhere; that gap is the open-challenge case, and it is stated rather than papered over (`docs/19_POLICY_ANCHOR.md` §7).

Repository: github.com/adeviwon/SightLine · commit `2e12860` (`refs/heads/pwa`) · 2026-10-22 · SightLine, Imperial Hackathon Hong Kong & Macau 2026.

Numbers in §2 were produced by the repository's own scoring functions (`ml/src/bench_hackathon.py`, imported rather than re-implemented) from the `artifacts/pipeline_raw*.json` files named beside every row — the shipped row's `pipeline_raw_shipped.json` is the current fixture generation, and every earlier row is labelled with its generation; the command that reproduces them is in §5. Numbers in §3, §4 and §7 are quoted from the file named beside them. Anything we could not trace to a file in this repository is not on this sheet.